

COMP1531

Software Engineering Fundamentals

Week 4



Development - Advanced Functions

Lecture Objective:

— — —

- Explore powerful ways higher-level languages use functions
- Learn **function syntax** and structure
- Explore **first-class** and **higher-order** functions
- Understand the role and use of **callbacks**



Function Syntax

In Javascript there are three equivalent ways to define functions.

Method 1

```
1 function sum(a, b) {  
2   return a + b;  
3 }
```

`fn_syntax_1.js`

Method 2

```
1 const sum = function(a, b) {  
2   return a + b;  
3 };
```

`fn_syntax_2.js`

Method 3

```
1 const sum = (a, b) => {  
2   return a + b;  
3 };
```

`fn_syntax_3.js`

- Method 1 is the old school method. Though it's fine :)
- Method 2 and 3 are similar, treating functions like variables.
- Method 3 is more modern and has other advantages.



Function Syntax

In Javascript there are three equivalent ways to define functions.

Method 1

```
1 function sum(a: number, b: number) {  
2   return a + b;  
3 }
```

`fn_syntax_1.ts`

Method 2

```
1 const sum = function(a: number, b: number) {  
2   return a + b;  
3 };
```

`fn_syntax_2.ts`

Method 3

```
1 const sum = (a: number, b: number) => {  
2   return a + b;  
3 };
```

`fn_syntax_3.ts`

- Method 1 is the old school method. Though it's fine :)
- Method 2 and 3 are similar, treating functions like variables.
- Method 3 is more modern and has other advantages.



Function Syntax

Method 3 also has another advantage. IFF (If and only if) the function body is a single line body that simply returns a value, the braces { } and return keyword can be omitted.

Method 3

```
1 const sum = (a: number, b: number) => {  
2   return a + b;  
3 };
```

[fn_syntax_3.ts](#)

Method 3 Compact

```
1 const sum = (a: number, b: number) => a + b;  
2  
3 // so short!!
```

[fn_syntax_3_compact.ts](#)



Function Syntax

Another example: You might have previously written this function like this:

Many String

```
1 function manyString(repeat: number, str: string) {  
2   let outString = '';  
3   for (let i = 0; i < repeat; i++) {  
4     outString += str;  
5   }  
6   return outString;  
7 }  
8 console.log(manyString(5, 'hello '));
```

`many_string_1.ts`

Many String - Advanced

```
1 const manyString = (repeat: number, str: string) => {  
2   let outString = '';  
3   for (let i = 0; i < repeat; i++) {  
4     outString += str;  
5   }  
6   return outString;  
7 };  
8 console.log(manyString(5, 'hello '));
```

`many_string_3.ts`

First Class Functions

— — —

In JavaScript, functions are **treated like variables** → can be **stored, passed, or returned**. (This is a language feature.)

```
// ✅ First-Class Function
function greet(name) {
  return `Hello, ${name}!`;
}

const sayHi = greet; // Function assigned to a variable ← first-class property
console.log(sayHi("Shaveen")); // "Hello, Shaveen!"
```

```
// ✅ First-Class Function (Expression)
const greet = function(name) {
  return `Hello, ${name}!`;
};

const sayHi = greet; // Function assigned to a variable ← first-class property
console.log(sayHi("Shaveen")); // "Hello, Shaveen!"
```

First Class Functions

— — —

You're already used to the idea that you can write a function that produces a different output based on the variable inputted.

```
const say_Hi = (name: string) => {  
  | return `Hello ${name}!`;   
};  
console.log(say_Hi('Shaveen'));
```

fcf_var.ts

First Class Functions

So let's expand that idea and say we can write a function that produces a different output based on a function that is inputted. We essentially pass the function object in, and then call the function inside the body

```
type Fmtr = (str: string) => string;

const brackets = (str: string) => `(${str})`;
const fullstop = (str: string) => `${str}.`;
const sayHi = (name: string, format: Fmtr) => `Hello, ${format(name)}!`;

const result = sayHi('COMP1531', brackets) +
  ' -- ' +
  sayHi('Shaveen', fullstop);

console.log(result);
```

[fcf_format_atomic_alt.ts](#)

First Class Functions - Anonymous Functions

— — —
If we only intend to use a function once, we can pass it directly into a function.
Because this function doesn't have a name, we call it an anonymous function.

```
type Fmtr = (str: string) => string;

const names = ['Henry', 'Shaveen', 'Yuchao'];

function formatNames(list: string[], format: Fmtr) {
  const newList: string[] = [];
  for (const name of list) {
    newList.push(format(name));
  }
  return newList;
}

console.log(formatNames(names, (str: string) => `${str}`));
console.log(formatNames(names, (str: string) => str.toLowerCase()));
console.log(formatNames(names, (str: string) => `[${str}]`));
```

fcf_format_loop_new.ts

```
type Fmtr = (str: string) => string;

const names = ['Henry', 'Shaveen', 'Yuchao'];

function formatNames(list: string[], format: Fmtr) {
  const newList = [];
  for (const name of list) {
    newList.push(format(name));
  }
  return newList;
}

const newNames = formatNames(names, (str) => {
  return `${str}`;
});

console.log(newNames);
```

fcf_format_loop_anon.ts

First-Class Functions

— — —

In Summary: What

- **First-class functions** mean functions are treated like variables.
- They can be **passed as arguments**, **returned**, or **stored**.
- This enables functions to **accept other functions** as inputs.

Why ?

- Enables **concise and readable code**.
- Supports **anonymous (one-off) functions** for cleaner logic.
- Forms the foundation for **callbacks**, **higher-order functions**, and **functional programming**.

Map, Reduce, Filter

— — —

Map, reduce, filter are functions that act on array objects that help us accomplish basic iterative tasks without the overhead of a loop setup. They are based on ideas of firstclass functions and anonymous functions

- Map: Modify an array
- Filter: Select from an array
- Reduce: Summarise an array

Map, Reduce, Filter

— — —

Map

- Takes an array of size **N**, and produces a new array of size **N** having modified each element according to a function passed in.
- Same array size. Modified elements.

Map, Reduce, Filter

Map

Classic

```
1 const tutors = [  
2   'Sandeep',  
3   'Chaw',  
4   'Rani',  
5   'Junya',  
6 ];  
7  
8 const loudTutors = tutors.map((str: string)  
9   return `${str.toUpperCase()}!!!`;  
10  });  
11  
12 console.log(loudTutors);
```

map_old.ts

Modern

```
1 const tutors = [  
2   'Sandeep',  
3   'Chaw',  
4   'Rani',  
5   'Junya',  
6 ];  
7  
8 const shout = function(str: string) {  
9   return `${str.toUpperCase()}!!!`;  
10 };  
11  
12 const newList = tutors.map(shout);  
13 console.log(newList);
```

map.ts

Same array size. Modified elements.

Map, Reduce, Filter

— — —

Filter

- Takes an array of size **N**, and produces a new array of size **0..N** without modifying any of the data.
- Possibly smaller array size. Elements unchanged.

Map, Reduce, Filter

Filter

Classic

```
1 type Student = {
2   name: string;
3   mark: number;
4 }
5
6 const students = [
7   {
8     name: 'Ben',
9     mark: 45,
10  },
11  {
12    name: 'Yuchao',
13    mark: 75,
14  },
15  {
16    name: 'Giuliana',
17    mark: 85,
18  },
19  {
20    name: 'Rani',
21    mark: 85,
22  },
23 ];
24
25 const newList = students.map((student: Student) => student.mark >
26
27 console.log(newList);
```

[filter_old.ts](#)

Modern

```
1 const marks = [65, 72, 81, 40, 56];
2
3 const isPass = function(mark: number) {
4   return mark >= 50;
5 };
6
7 const newList = marks.filter(isPass);
8 console.log(newList);
```

[filter.ts](#)

Possibly smaller array size. Elements unchanged.

Map, Reduce, Filter

— — —

Reduce

- Executes a reducer function (that you provide) on each member of the array resulting in a **single output** value.
- Array turned into atomic value.

Map, Reduce, Filter

Reduce

Classic

```
const students = [
  { name: 'Amy', mark: 55 },
  { name: 'Bob', mark: 43 },
  { name: 'Cap', mark: 34 },
  { name: 'Dex', mark: 23 },
];

let single = 0;
for (const student of students) {
  single += student.mark;
}
console.log(single);
```

reduce_old.ts

Modern

```
type Student = {
  name: string;
  mark: number;
}

const students: Student[] = [
  { name: 'Amy', mark: 55 },
  { name: 'Bob', mark: 43 },
  { name: 'Cap', mark: 34 },
  { name: 'Dex', mark: 23 },
];

const sum = (prev: number, curr: Student) => {
  return prev + curr.mark;
};

const single = students.reduce(sum, 0);
console.log(single);
```

reduce.ts

Array turned into atomic value.

Map, Reduce, Filter - Anonymous Functions

We can use anonymous functions to further streamline our previous code.

```
1 const tutors = ['Simon', 'Teresa', 'Kaiqi', 'Michelle'];
2 const newList = tutors.map((string) => `${string.toUpperCase()}!!!`);
3 console.log(newList);
```

map_stream.ts

```
1 const marks = [65, 72, 81, 40, 56];
2 const newList = marks.filter((mark) => mark >= 50);
3 console.log(newList);
```

filter_stream.ts

```
1 const students = [
2   { name: 'Amy', mark: 55 },
3   { name: 'Bob', mark: 43 },
4   { name: 'Cap', mark: 34 },
5   { name: 'Dex', mark: 23 },
6 ];
7 const single = students.reduce((prev, curr) => prev + curr.mark, 0);
8 console.log(single);
```

reduce_stream.ts

Map, Reduce, Filter - Combined Example

— — —



`map_reduce_filter_demo.ts`

Higher Order Functions

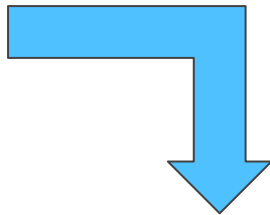
— — —

- Higher-order Functions are essentially functions that return functions. Think of them like mini function factories.
- Let's go through a code cleanup journey.

Higher Order Functions - Another Example

```
function congratMarkPS(name: string) {  
  return `Congratulations ${name} on your pass`;  
}  
function congratMarkCR(name: string) {  
  return `Congratulations ${name} on your credit`;  
}  
function congratMarkDN(name: string) {  
  return `Congratulations ${name} on your distinction`;  
}  
console.log(congratMarkCR('Shaveen'));
```

hoc_1.ts



```
const genCongratMark = (markstr: string) => (name: string) =>  
  `Congratulations ${name} on your ${markstr}`;  
  
const congratMarkPS = genCongratMark('pass');  
const congratMarkCR = genCongratMark('credit');  
const congratMarkDN = genCongratMark('distinction');  
  
console.log(congratMarkCR('Shaveen'));
```

hoc_7.ts

Higher Order Functions

— — —

Summary

- Higher order functions are syntactically sleek ways to generalise function definitions.
- It make code **reusable, flexible, and expressive**, allowing developers to write **cleaner, modular, and more powerful** JavaScript programs.

Feedback

COMP1531 Feedback Survey

